

注意力机制

想法很简单：从含量很大的输入信息中"注意"到真正有用的信息是很有必要的。

对于人类，有两种原因会导致注意力聚焦在某个事物上：

1. **非自主性提示**：指由于某个事物在一堆事物中很"显眼"，让我们的注意力集中。
2. **自主性提示**：指由于某个想法或意识的指示去注意某个事物。

举例：三样物品——黑白印刷的书，空白笔记本，一杯咖啡（杯子是显眼的红色）。

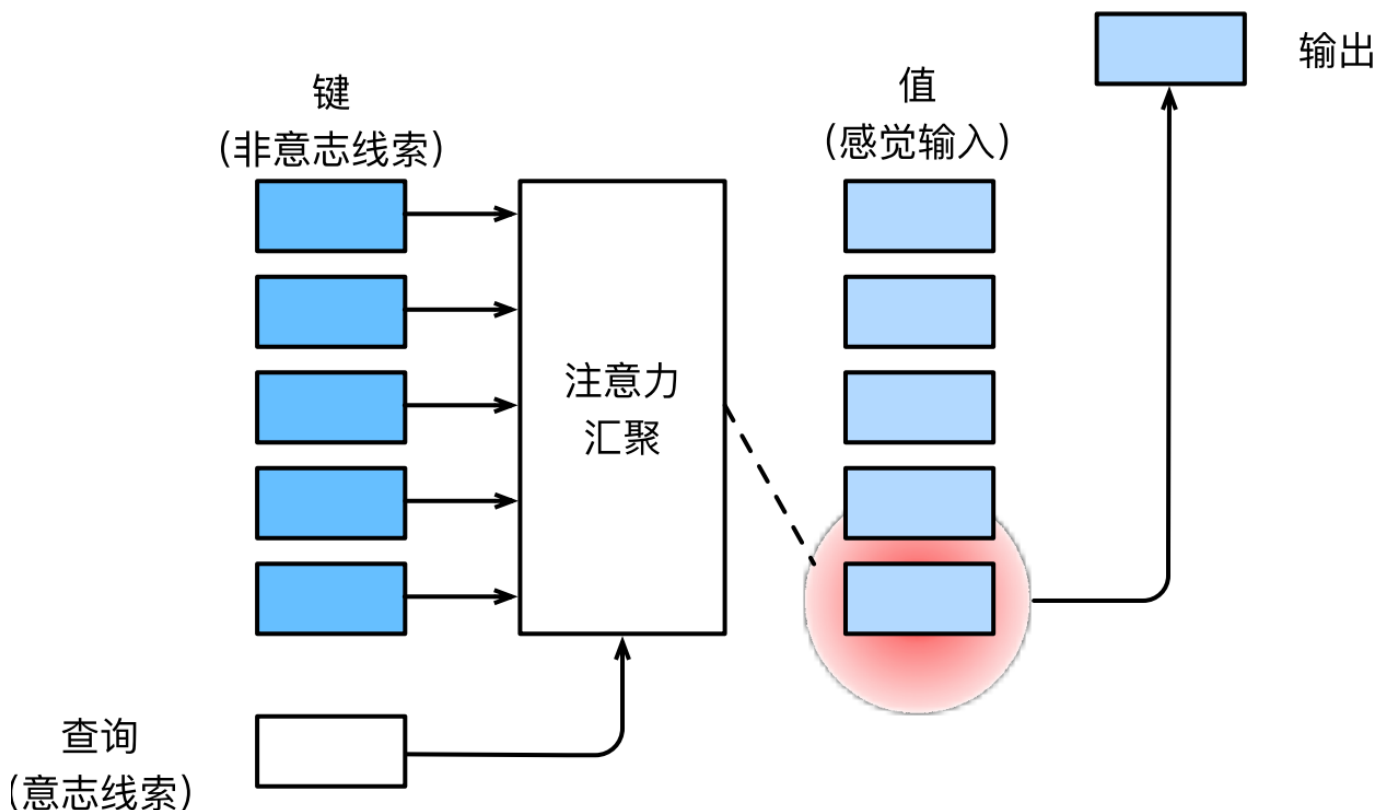
- 咖啡由于杯子颜色鲜艳吸引了我们的注意力（非自主性提示）。
- 喝完咖啡后文思泉涌，此时我们想找能写字的本子，注意到了空白笔记本（自主性提示）。

对于机器：非自主性提示很简单（甚至我们之前就已经实现过了），通过全连接层 / 汇聚层，我们可以"筛选"出"显眼"的特征。所以下面主要探讨自主性提示。

三个概念

- **Query (查询)**：实现自主性提示的关键，即"我想要找什么"。
- **Key (键)**：非自主性提示，即每个候选信息的特征。
- **Value (值)**：真正要被提取的信息。其中，键与值一一对应。

核心思想：给定 query，根据各个 key 与 query 的匹配程度，给其对应的不同 value 分配不同权重，即实现了不均匀的注意力分配。如下图所示（图中的输出不重要，根据具体任务可以是选出权重最大的 value，也可以是对 value 根据权重加权求和）。



【图片：注意力汇聚示意图 —— 键（非意志线索） / 值（感觉输入） / 查询（意志线索） → 注意力汇聚 → 输出】

注意力汇聚：Nadaraya-Watson 核回归

这一节我们做一个具体的事情：给定若干由函数 $y = f(x) + \epsilon$ （噪声 ϵ ）生成的数据集 $\{(x_1, y_1), (x_2, y_2), \dots\}$ ，预测当输入 x 时的输出 $\hat{y}$ 。

（这个任务可以用最简单的多层感知机直接很好地解决）这里我们用注意力汇聚来解决这个问题，即 Nadaraya-Watson 核回归。

首先看如果注意力不汇聚会怎么样：注意力不汇聚 → 对这些输入没有特定倾向 → 对所有给定数据都一视同仁 → 预测为所有 y 的平均（即平均汇聚）。这很显然不够好。

所以，引入**注意力汇聚**：对输入的数据组，根据其 x 进行加权，再将对应的 y 加权求和得到 $\hat{y}$ 。很显然的想法是：离我要求的 x 更近的数据点更具有参考价值。于是有：

$$f(x) = \sum_{i=1}^n \alpha(x, x_i) y_i$$

其中 $f(x)$ 就是预测的 $\hat{y}$ 。

注意力机制三概念也都体现在这个公式里：

- x ：查询 query，也就是当前要预测的位置。
- x_i ：键 key，也就是训练样本的输入。
- y_i ：值 value，对应 x_i 的输出。
- $\alpha(x, x_i)$ ：即注意力权重，其计算公式如下。

$$\alpha(x, x_i) = \text{softmax} \left(-\frac{1}{2}(x - x_i)^2 \right)$$

这表示，对于查询 x ，离 x 越近的数据点权重越高。以上是**非参数注意力汇聚**，也就是说无需训练，给定输入直接就能预测输出。

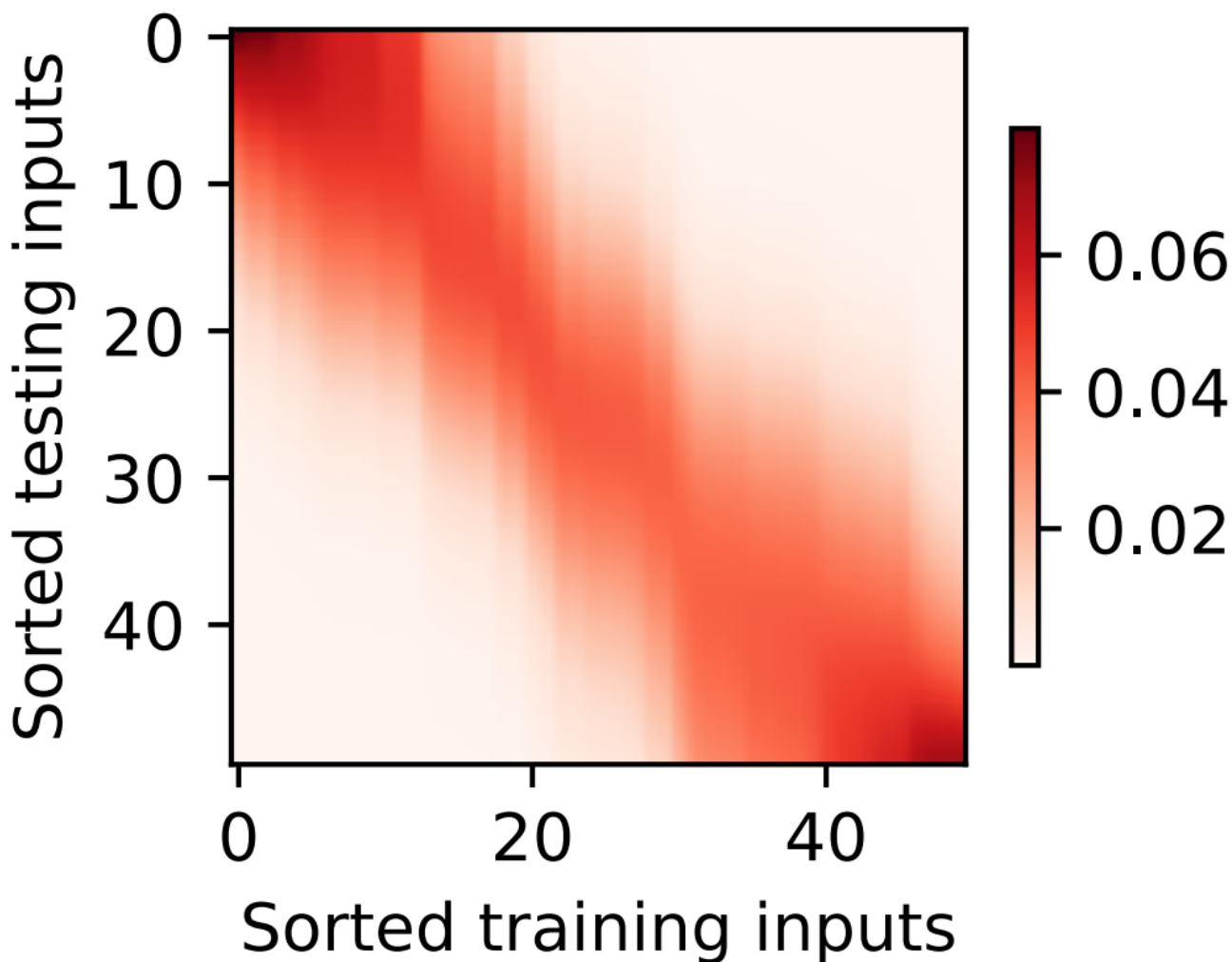
升级版：带参数注意力汇聚——添加一个可学习参数 w ，用来控制注意力分布的"宽窄"。公式为：

$$\alpha(x, x_i) = \text{softmax} \left(-\frac{1}{2}((x - x_i) w)^2 \right)$$

w 的大小	效果
w 小	距离影响较弱，注意力更分散，曲线更平滑
w 大	距离影响更强，注意力更集中，曲线更尖锐

所以 w 类似于一个可学习的"带宽控制器"。模型可以通过训练自动学习：预测时到底应该看多大范围内的邻居。

一个很重要的图：**注意力权重图**，如下所示。

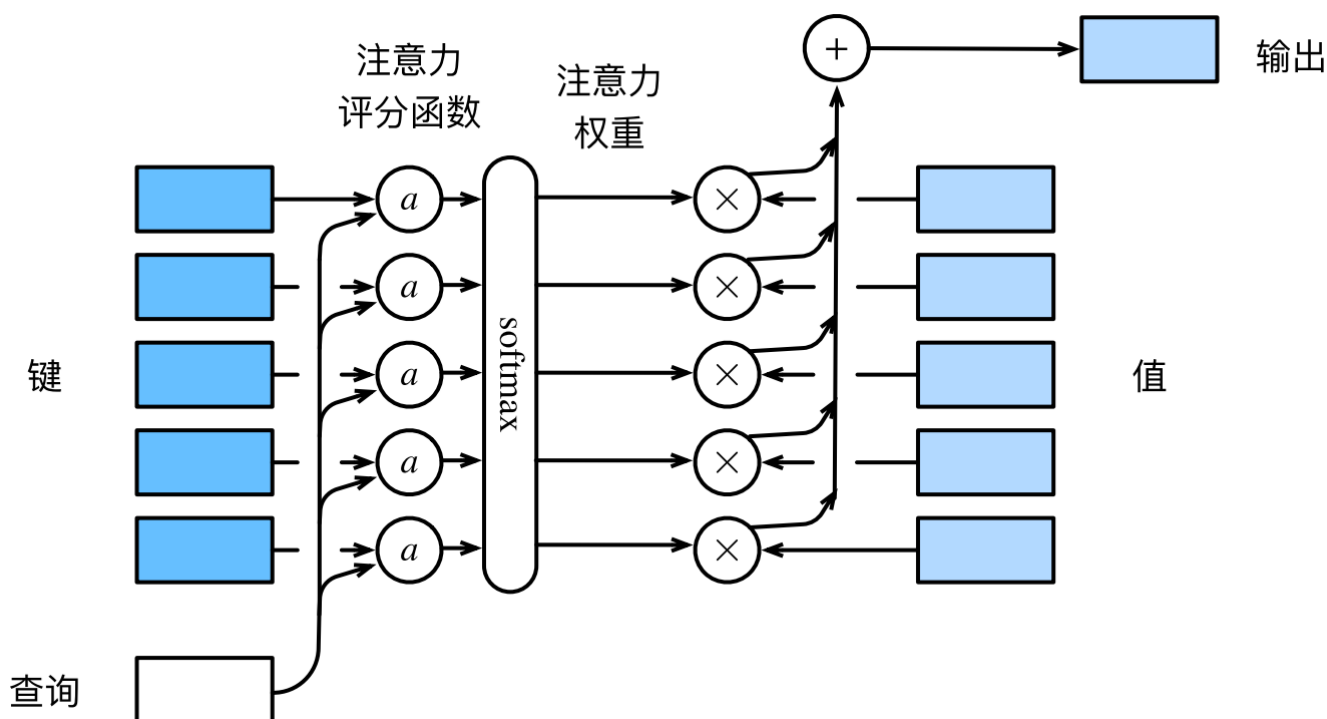


【图片：注意力权重热力图 —— 横轴 Sorted training inputs, 纵轴 Sorted testing inputs, 对角线颜色深】

- 横轴、纵轴分别是 x_i 、 x 从小到大**排序的序号**（注意不是它们的真实值）。
- 对应像素的颜色深浅代表注意力权重，即 $\alpha(x, x_i)$ 。
- 该图特点：对角线上权重大，其余部分权重小。
- 原因：对角线上 x 和 x_i 相对接近，权重高，颜色深。

注意力评分函数

上面一节是一个最简单的例子，我们从这个例子中抽象出普适的注意力机制框架。



【图片：注意力评分函数框架图 —— 键 + 查询 → 注意力评分函数 $a()$ → softmax → 注意力权重 → (与值加权) → 输出】

流程：查询和每个键组队，经过注意力评分函数得到每个键值对的分，经过 softmax 归一化为概率（注意力权重），根据这个权重对值进行加权求和得到输出。

可以看出，其中除了注意力评分函数，其他都是 trivial 的。本节重点关注注意力评分函数。注意力评分函数符号表示为 $a(\cdot)$ ，函数的输出分数为标量。

有三个要注意的地方：

1. 掩蔽 softmax 操作：在序列中，有两类模型不应该去注意的位置——填充词元（即现代循环神经网络里面讲到的 `<pad>`）和未来词元（生成第三个词元时，不应该看到 4、5 等后面的）。为了“掩蔽”这些位置使其不被注意，做法是：在 softmax 之前，将这些位置的分调整为很大的负数（例如 -10^6 ），这样在 softmax 中权重几乎为 0，实现“掩蔽”。

2. 加性注意力：query 和 key 的维度不一致时使用。记为：

$$a(\mathbf{q}, \mathbf{k}) = \mathbf{w}_v^\top \tanh(\mathbf{W}_q \mathbf{q} + \mathbf{W}_k \mathbf{k}) \in \mathbb{R}$$

其中 $\mathbf{q}$ 为 query 查询， $\mathbf{k}$ 为 key 键， $\mathbf{W}_q$ 、 $\mathbf{W}_k$ 、 $\mathbf{w}_v$ 都是可学习参数。也就是说，处理方法是先将 query 和 key 通过多层感知机揉到同一个维度去，然后再用 tanh 归一化，乘以向量变成标量。

3. 缩放点积注意力：query 和 key 的维度一致，此时可以直接算点积。

$$a(\mathbf{q}, \mathbf{k}) = \mathbf{q}^\top \mathbf{k} / \sqrt{d}$$

其中 d 是 query 和 key 的长度。除以 $\sqrt{d}$ 的目的是防止点积数值过大导致不稳定。

Bahdanau 注意力

实际问题：比如机翻，当翻译的句子过长时，容易“抓不住重点”。

例如：翻译 *The student who borrowed my book yesterday is waiting outside the library.*

生成不同内容时应当注意的源内容也是不同的，而传统 seq2seq 中，源内容经过编码器只得到了一个固定的上下文变量 c （包含了原句中的所有内容，但是没有具体细节部分的指向）。（换句话说讲，解码器不知道在当前翻译时该看哪一部分。）

快速回顾 seq2seq 流程：原内容经编码器处理生成 c ，解码器每一步获得三个输入——固定的 c 、上一步隐状态 s_t 、上一步生成的结果 y_t ，根据这三个输入生成下一步隐状态 s_{t+1} 和下一步预测结果 y_{t+1} ，循环往复。

Bahdanau 注意力的改进

编码器方面（底层改进，提供更多信息）：编码器每读入一个 token，就根据**当前已读的所有 token** 生成一个隐状态 h_i （也就是说 h_t 包含了原输入前 t 个 token 的信息）。

解码器方面（实现注意力机制）：解码到第 t 步时，有以下注意力机制：

- **查询 query**：上一步解码器的隐状态 s_{t-1} 。
- **键 key**：所有“参选”的编码隐状态 $h_1, h_2, \dots, h_i$ 。
- $\rightarrow$ 得到注意力权重（对于不同的编码隐状态，应当给予多大的注意力权重）：

$$\alpha_{ti} = \text{softmax}(a(s_{t-1}, h_i))$$

- **值 value**：也是 h_i （这里注意 h_i 同时作为键和值：作为键，它们和 query 匹配；作为值，它们自身就是“查询”的结果）。

得到每一步的 c_t ：不同编码隐状态 $\times$ 对应权重求和：

$$c_t = \sum_i \alpha_{ti} h_i$$

在这里即为当前的上下文。

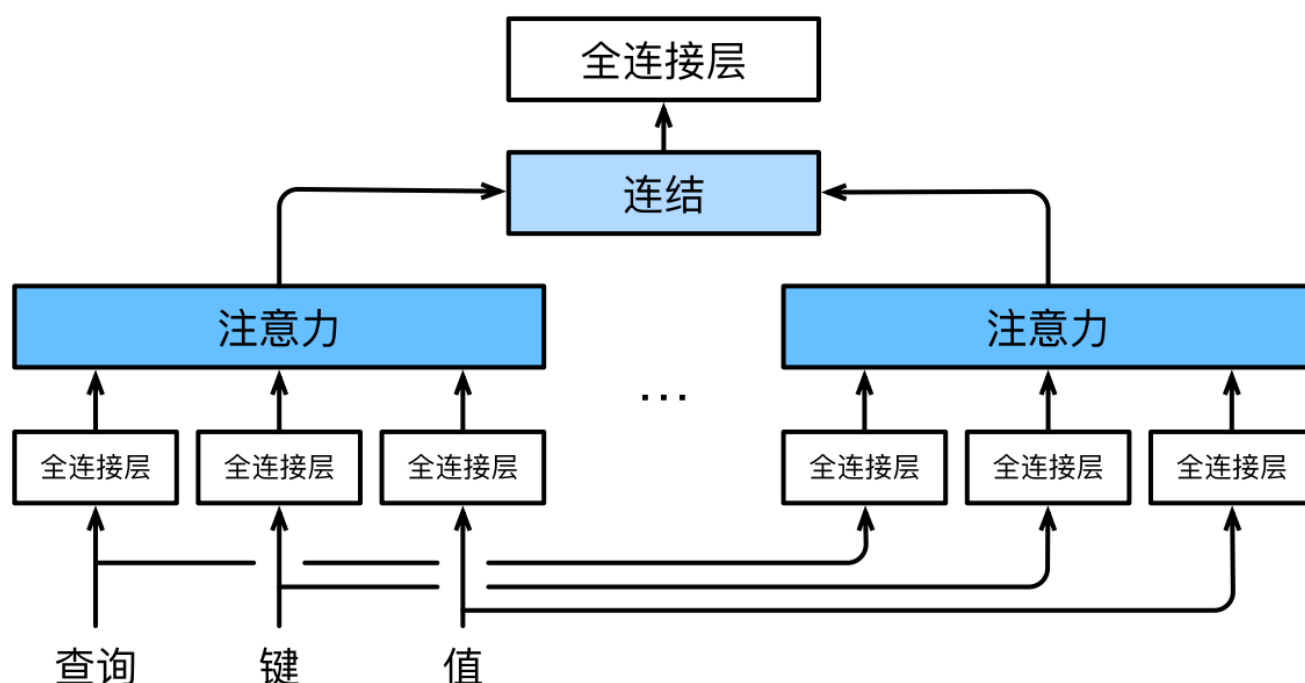
到这里我们已经得到了解码器每一步所需的 c, s, y ，其余类似 seq2seq 处理即可。

其实 s, y 没有任何变化，只是每一步的 c 要根据上一步的隐状态 s_{t-1} 重新算一遍。

多头注意力

(类似现代卷积神经网络中的含并行连结的网络 GoogLeNet)

核心思路：对于同一组查询和键值对，通过不同的角度进行注意力汇聚，最终得到结果。



【图片：多头注意力结构图 —— 查询/键/值 → 各自的全连接层 → 多个注意力 → 连结 → 全连接层】

回顾含并行连结的网络（GoogLeNet）：对于同一个输入，用不同尺度的卷积层来处理，最后合并得到处理结果。

多头注意力处理流程：对于同样的查询和键值对，经过独立的、不同的全连接层处理，投影到不同的空间上去，得到不同的查询和键值对；然后每一组查询和键值对再由一般的注意力汇聚得到结果，合并得到最终结果。

为什么说是投影？实际上，全连接层训练的参数是不同的矩阵，对于同样的查询和键值对，用不同的矩阵处理就是在不同的空间上进行投影。

自注意力和位置编码

自注意力： 查询、键和值都来自同一组输入。

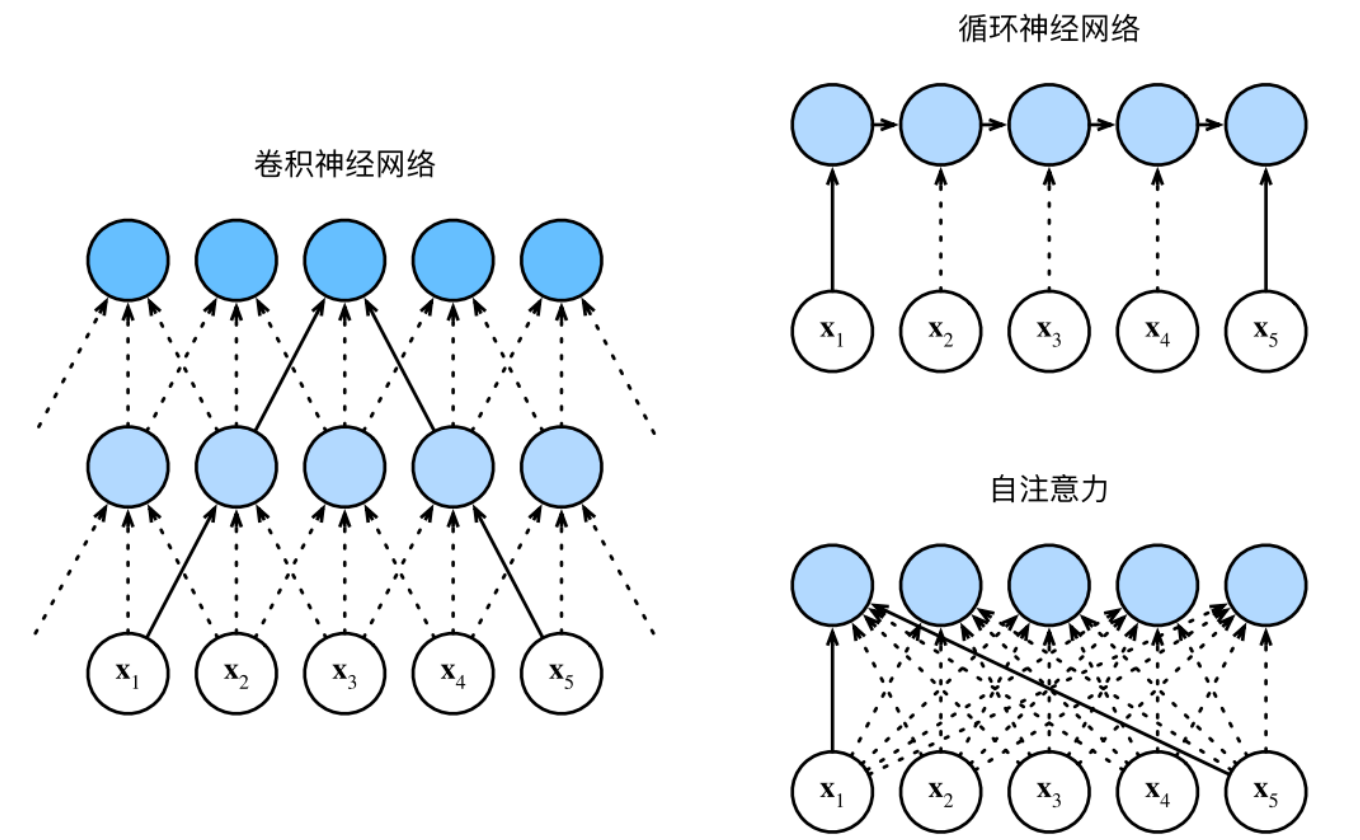
假设输入为 x_1, x_2, x_3 （都是 token），则自注意力的输出为 y_1, y_2, y_3 ，其中 y_i 是以 x_i 为查询，对整个输入序列（这里是 x_1, x_2, x_3 ）进行注意力汇聚得到。

也就是：

$$y_1 = \sum_i x_i \cdot \text{softmax}(a(x_1, x_i))$$

y_2, y_3 同理。

比较 CNN、RNN 和自注意力



【图片：CNN / RNN / 自注意力 三种结构对比图】

总结总表如下：

架构	计算复杂度	顺序操作	最大路径长度	特点
CNN	$O(knd^2)$	$O(1)$	$O(n/k)$	可并行，但远距离交互需要多层
RNN	$O(nd^2)$	$O(n)$	$O(n)$	顺序建模强，但难并行
自注意力	$O(n^2d)$	$O(1)$	$O(1)$	全局交互强，可并行，但长序列代价高

计算复杂度： 对于一个长度为 n 、每个 token 为 d 维的序列，需要多少次计算？

- **CNN：** 卷积核大小为 k 。对于一个 token，它看到了 k 个相邻的 token，每个 token 有 d 个维度，所以输出每个数字的计算次数为 kd ，输出共 d 维，故一个 token 对应 kd^2 ， n 个 token 即为 nkd^2 。
- **RNN：** 和 CNN 类似，但是一个 token 只能看到前一个隐状态， $k = 1$ 。
- **自注意力：** 和矩阵的运算有关，结果为 n^2d 。

顺序操作： 对于长度为 n 的序列，全部计算完毕需要多少时间步？

- **CNN、自注意力：** 计算中没有“顺序”的说法，每个 token 的计算和其他 token 无关，无依赖关系，可以同时进行，故顺序操作 $O(1)$ 。
- **RNN：** 有“顺序”，后面的 token 计算依赖于前面 token 计算得到的隐状态，无法并行计算，一次只能算一个，故顺序操作 $O(n)$ 。

最大路径长度： 对于长度为 n 的序列， x_1 与 x_n 建立联系要多少步？

这个问题可以看图理解，图中实线箭头就表明首尾两个 token 建立了联系。

- **CNN：** 每一层卷积使感受野扩大 k ，总规模 n ，故需要 $O(n/k)$ 步使首尾建立联系。
- **RNN：** 一次一步，必须从头走到尾， $O(n)$ 。
- **自注意力：** 任意两个 token 都可以直接一步连接， $O(1)$ 。

位置编码

为什么需要？ 自注意力中，所有 token 的计算并行进行，使得其不知道 token 之间的相对位置关系，所以需要人为地根据 token 的位置对 token 进行位置编码，**指导机器认识位置关系**。

做法： 输入矩阵 X 是 $n \times d$ (n 个 token，每个 token 有 d 维)，对应的位置编码矩阵 P 也是 $n \times d$ ，和 X 一一对应。向后**输出为** $X + P$ (正常矩阵加法)。

P 的确定： 用正余弦函数，公式如下：

$$p_{i, 2j} = \sin\left(\frac{i}{10000^{2j/d}}\right)$$

$$p_{i, 2j+1} = \cos\left(\frac{i}{10000^{2j/d}}\right)$$

其中 $p_{a,b}$ 为 P 矩阵第 a 行第 b 列的元素值。

具体细节省去，这样做的好处：机器需要**学习 token 之间的相对位置**，用这种编码方式可以使得由 i 位置推导 $i + \delta$ 位置只需要乘一个和 i 无关的矩阵就可以，便于机器学习相对位置。（具体证明用到三角恒等变换，此处省去。）

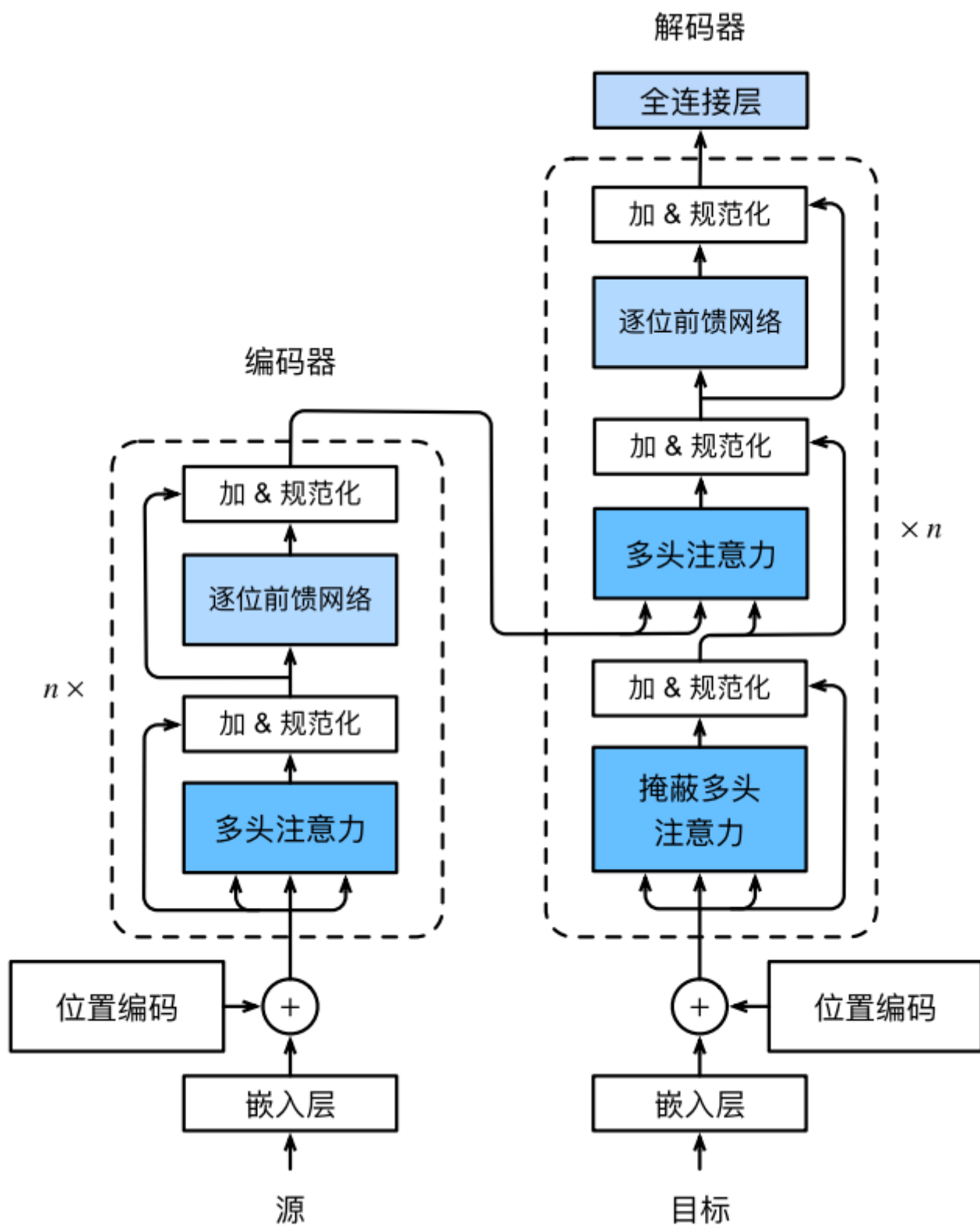
简单来讲，我知道了第 i 个 token 的位置信息，我只需要乘以一个只和 δ 有关的矩阵就可以得到第 $i + \delta$ 个 token 的位置信息。

其他好处：

- **扩展性强**（ i 和 j 不管是多少都可以算）。
- **稳定性好**（值都在 -1 到 1 之间，不会过大）。

Transformer

是什么？ 和 CNN（以卷积为核心）、RNN（以循环状态递推为核心）类似，是**以自注意力为核心的神经网络架构**。一个 Transformer 架构包括 **encoder 和 decoder**（编码器和解码器），示意图如下所示。



注意：该架构是经典的 Transformer 架构，是完整的 encoder 和 decoder 的组合，应用于 seq2seq 的预测。实际中也有单独使用其 encoder 或 decoder 的。

【图片：图 10.7.1 Transformer 架构 —— 左边 encoder，右边 decoder】

左边是 encoder，右边是 decoder。

Encoder

由于自注意力没有"位置"的概念，所以需要加上**位置编码**（参考"位置编码"一节）。然后输入 n 个**串联的 encoder block**，每个 encoder block 里面有一个**多头注意力**模块和一个**逐位前馈网络**，模块之间使用**残差连接**。

注意：由于残差连接的要求，输入经过模块处理后的输出必须和输入的**形状相同**。因此，**前一个 encoder block 的输出直接作为下一个 encoder block 的输入**。（图中不经过处理、直接绕过去的箭头和"加"就是残差连接。）

Encoder block **内部处理**：

1. **多头注意力**：将不同 token 中的信息相混合；多头 $\rightarrow$ 让不同头关注不同关系。
2. **逐位前馈网络**：对于每个 token，都用**同一套参数**做一次 MLP。

简单理解：多头自注意力让不同的 token 之间交流信息，逐位前馈网络对每个 token 自己的特征做加工。

Encoder **最终交付的是什么**？是和输入 token 形状一模一样的隐状态（上下文）。例如，输入 token 为 x_1, x_2, x_3 ，则输出为 h_1, h_2, h_3 ，其中 h_i 为 x_i **在输入中的上下文信息**。

Decoder

也是由 n 个**串联的 decoder block** 拼接而成，每个 decoder block 里面有三个模块。

模块与模块之间也是采用**残差连接**，**所以形状不变**，前一个块的输出直接作为后一个块的输入，和 encoder 里面很类似。

1. **掩蔽多头注意力**：（掩蔽即生成第 i 位时只能看前面的、不能看后面的，**只在训练时起作用**：训练时直接给 decoder 完整的生成序列，而预测时只能一步一步预测。）通过**多头自注意力**的方式将已生成的 token 序列转化为已生成序列的信息传递（这里记为 $y_1, y_2, \dots$ ）给下一级。
2. **多头注意力**：（注意：这里是 encoder 信息的输入口，且**不是自注意力**了。）encoder 将 $h_1, h_2, \dots$ 提供给 decoder，在这里进行多头注意力：**query 为 $y_1, y_2, \dots$ ，key 和 value 都是全部的 h** 。

换句话讲， y_1 经过多头注意力的输出是：{以 y_1 为 query，全部的 h 为 key 和 value 进行多头注意力得到的结果}， y_2 等同理。

3. **逐位前馈网络**：和 encoder 中类似，也是对于各位 token 分别进行信息特征加工，而不是 token 之间互相交流。

n 层 decoder 块跑完以后，输出结果经过全连接层、softmax 输出为下一位预测各 token 的概率，从而决定下一位输出什么。

规范化（层规范化）

对所有的 token，每个 token 里面所有维度的数字进行归一化，然后做可学习的缩放和偏移，称为**层规范化**。

归一化：算该 token 里所有数字的均值和方差，然后归一化。

例子：假设某个 token 的 hidden vector 是：

$$x = [2, 4, 6, 8]$$

均值是：

$$\mu = \frac{2 + 4 + 6 + 8}{4} = 5$$

每一维减去均值：

$$[-3, -1, 1, 3]$$

标准差大约是：

$$\sqrt{5}$$

所以规范化后大概是：

$$[-1.34, -0.45, 0.45, 1.34]$$

效果：把 token 里面的隐藏向量调整到更稳定的数值范围，使训练结果稳定。

注意：这里的层规范化和之前的批量规范化（参考“现代卷积神经网络”笔记）不一样。批量规范化是以一个 batch 里的多个 token 作为样本进行规范化，而这里的层规范化是以单个 token 作为样本，在其内部做规范化。